

Informe de Proyecto: Detección y Mapeo Georreferenciado de Residuos mediante Enjambre UAV

Hugo Sevilla Martínez

DNI: [48798351X]

Hugo López Pastor

DNI: [53979557Y]

Grado en Ingeniería Robótica

Índice

1. Objetivos	2
2. Software Empleado	2
3. Manual de Ejecución	2
4. Desarrollo Técnico	3
4.1. Configuración del Entorno Físico (Gazebo)	3
4.2. Optimización del Modelo URDF mediante XACRO	3
4.3. Percepción: Integración de Sensores	4
4.4. Arquitectura Multi-Robot y Control Distribuido	5
4.5. Visión Artificial y Posicionamiento GPS	7
5. Estructura del Sistema de Ficheros	7
6. Dificultades y Resoluciones	8
7. Conclusiones y Trabajo Futuro	9
8. Uso de Herramientas de IA	9

1. Objetivos

El objetivo de este proyecto es diseñar, modelar y simular un enjambre de drones (UAVs) coordinados para detectar y mapear basura acumulada en las playas. Queremos crear un entorno de simulación realista usando ROS 2 y Gazebo Harmonic, integrando los modelos físicos de los drones, sus sensores, visión artificial con YOLOv8 y un algoritmo para calcular la posición real de cada residuo. Con esto, buscamos automatizar la búsqueda de basura y crear un mapa que sirva para facilitar las tareas de limpieza.

2. Software Empleado

El desarrollo de este sistema se ha llevado a cabo sobre una máquina con Linux nativo utilizando las siguientes herramientas:

- **Ubuntu 24.04 LTS.**
- **ROS 2 Jazzy Jalisco.**
- **Gazebo Harmonic.**
- **RViz2** como herramienta de visualización.
- **Python 3** para los nodos de control, visión y consolidación de mapas.
- **XACRO** como preprocesador de archivos URDF.
- **Ultralytics (YOLOv8)** y **OpenCV** para el procesamiento de inferencia visual.
- **Matplotlib** para la generación dinámica del mapa de residuos.

3. Manual de Ejecución

Para desplegar la simulación completa, que permite lanzar de manera paramétrica múltiples drones interactuando coordinadamente en el entorno, se deben seguir estos pasos utilizando tres terminales independientes:

1. **Preparación del Entorno (Workspace):** Situándose en la raíz del workspace con los paquetes clonados (`uav_gazebo`, `uav_description`, `uav_control`, `uav_vision`), se deberá compilar e iniciar el entorno:

```
colcon build --symlink-install
source install/setup.bash
```

2. **Terminal 1: Lanzar el Entorno Físico (Gazebo):** Instancia el mundo con la textura de arena, los modelos 3D de residuos y el número deseado de robots (por defecto, $N = 3$).

```
ros2 launch uav_gazebo multi_uav.launch.py robots:=3
```

3. **Terminal 2: Lanzar la Inferencia Visual y Mapeo:** Arranca los nodos de detección YOLO, los nodos de georreferenciación individual y el consolidador global de mapa. El parámetro `show_gui:=false` suprime las ventanas de vídeo para optimizar el rendimiento de la CPU ya que hacen que la simulación vaya excesivamente lenta.

```
ros2 launch uav_vision multi_vision.launch.py robots:=3 show_gui:=false
```

4. **Terminal 3: Lanzar la Patrulla Sincronizada:** Inicia el algoritmo de barrido tipo *lawnmower*. Los drones despegarán, se dirigirán a sus puntos de inicio, esperarán la confirmación por red de todos sus compañeros y comenzarán la exploración simultánea.

```
ros2 launch uav_control multi_patrol.launch.py robots:=3
```

El resultado final se actualizará cada 10 segundos en los archivos `mapa_residuos.csv` y `mapa_residuos.png` generados en la raíz del `*workspace*`.

4. Desarrollo Técnico

Para implementar el sistema de monitoreo, se ha diseñado una arquitectura distribuida y modular donde cada robot opera de forma independiente y coordina sus esfuerzos con los demás de manera asíncrona. La Figura 1 detalla el flujo de información desde la simulación de sensores en Gazebo, pasando por el procesamiento individual en ROS 2, hasta la consolidación final en el mapa.

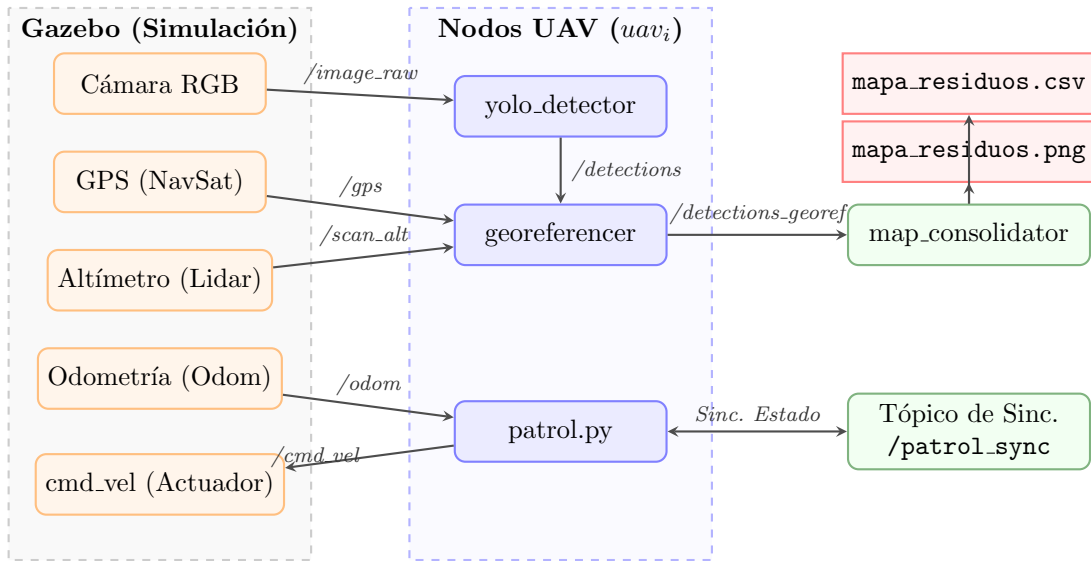


Figura 1: Diagrama de arquitectura de nodos ROS 2 y flujo de datos de simulación/procesamiento.

4.1. Configuración del Entorno Físico (Gazebo)

Para la simulación, empezamos diseñando un entorno exterior en el archivo `outdoor.world`. Para que la simulación fuese visualmente realista y la red neuronal pudiese detectar los residuos correctamente, configuramos un terreno de arena clara de alta resolución y colocamos diferentes tipos de basura fotorrealista a lo largo de la playa.

Usamos tanto modelos nativos descargados de Gazebo Fuel (como cajas de cartón o latas de refresco) como modelos 3D externos importados de internet para tener más variedad de formas y texturas (Figura 2). Tuvimos que ajustar a mano sus escalas, rotaciones y pesos en Gazebo para que se integraran bien en la arena y las físicas del simulador respondieran correctamente.



(a) Modelo externo: Botella de plástico.



(b) Modelo externo: Bolsa de snacks.

Figura 2: Ejemplo de modelos 3D externos de residuos integrados en la simulación.



Figura 3: Entorno de simulación en Gazebo (`outdoor.world`) con el enjambre desplegado sobre la zona de residuos modelados en 3D.

4.2. Optimización del Modelo URDF mediante XACRO

Como ya hicimos en las prácticas de clase, el modelado del dron se ha hecho con un archivo **URDF** estructurado mediante **XACRO** (XML Macros). En lugar de escribir un único archivo gigante y difícil de entender, dividimos el modelo en varios archivos modularizados:

- **Modelo Base** (`uav.xacro`): El archivo principal que une todo el chasis del dron (*base_link*), las hélices y llama al resto de ficheros.
- **Propiedades Comunes** (`common_properties.xacro`): Guarda las constantes, colores, texturas y las fórmulas para calcular las inercias de forma automática.
- **Sensores**: Archivos como `camera.xacro`, `altimeter.xacro`, `gps.xacro`, `lidar.xacro` e `imu.xacro` con sus respectivos sensores y plugins.
- **Actuador de Vuelo** (`flight_control.xacro`): Llama al plugin de Gazebo para mover los motores y que el dron responda físicamente al empuje.
- **Aislamiento por Prefijos**: Añadimos el argumento `prefix` para poder lanzar varios drones a la vez sin que choquen los nombres de los enlaces o tópicos en ROS 2 (por ejemplo, tener `uav1/base_link` y `uav2/base_link` por separado).

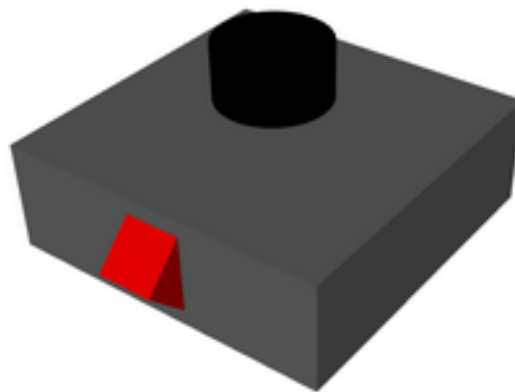


Figura 4: Visualización limpia del modelo cinemático URDF del UAV base en RViz2.

4.3. Percepción: Integración de Sensores

Cada dron lleva equipados cinco sensores en sus respectivos archivos `xacro`, que nos dan los datos necesarios para volar y detectar la basura:

- **Cámara RGB** (`camera.xacro`): Apunta hacia el suelo con una resolución de 640×480 píxeles y un campo de visión (HFOV) de 1.089 radianes. Sirve para capturar el vídeo que procesará la red neuronal.
- **Altímetro** (`altimeter.xacro`): Un sensor Lidar de un solo rayo apuntando hacia abajo. Mide la distancia al suelo, lo que nos ayuda a mantener el dron a 2.5 metros de altura de crucero y a hacer el cálculo de la georreferenciación.
- **NavSat / GPS** (`gps.xacro`): Sensor GPS que nos da la latitud y longitud global del dron, imprescindible para colocar los residuos en el mapa.

- **Lidar 360°** (`lidar.xacro`): Sensor láser en la parte superior del chasis para escanear obstáculos en plano, pensado para esquivar objetos en el futuro.
- **IMU** (`imu.xacro`): Unidad de medición inercial para medir aceleraciones y velocidades de giro, lo que ayuda a mantener estable el chasis del dron en el aire.

4.4. Arquitectura Multi-Robot y Control Distribuido

La lógica de navegación y coordinación se gestiona de forma totalmente descentralizada mediante el nodo `patrol.py`. Para que los drones no choquen y cubran toda la zona rápido, dividimos el área de la playa en franjas horizontales paralelas, asignando una a cada dron.

4.4.1. Subdivisión del Área y Generación de Trayectorias

La playa está acotada en el eje Y por el intervalo $[Y_{\min}, Y_{\max}] = [-7,0, 7,0]$ metros. La anchura w_y de la franja de cada dron se calcula de forma automática según los drones activos N :

$$w_y = \frac{Y_{\max} - Y_{\min}}{N} \quad (1)$$

Para un dron con identificador $i \in \{1, 2, \dots, N\}$, su zona asignada en el eje Y es:

$$Y_i \in [Y_{\min} + (i - 1)w_y, Y_{\min} + i \cdot w_y] \quad (2)$$

Dentro de su franja, cada dron realiza un barrido de ida y vuelta (tipo *lawnmower*) en el eje X (de 2,0 a 20,0 metros), con un paso lateral de $\Delta y = 3,0$ metros. El punto de inicio de la exploración se calcula en el centro de su primera pasada para que la cobertura sea perfecta:

$$y_{\text{start},i} = (Y_{\min} + (i - 1)w_y) + \min\left(\frac{\Delta y}{2,0}, \frac{w_y}{2,0}\right) \quad (3)$$

A partir de este punto, los waypoints se van incrementando sucesivamente en Δy alternando el sentido en el eje X. Al estar las franjas físicamente separadas, garantizamos que los drones patrullan en paralelo de forma segura y sin peligro de chocar.

4.4.2. Máquina de Estados y Protocolo de Sincronización

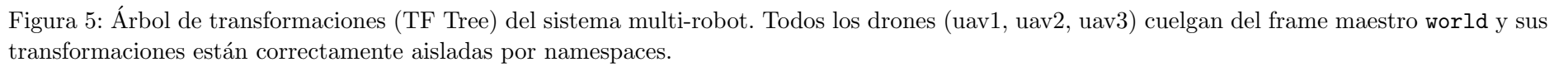
Para coordinar a los drones sin necesitar una estación central que les envíe órdenes, programamos en el nodo de control una máquina de estados para cada dron con las siguientes fases:

- **TAKEOFF**: El dron sube en vertical hasta llegar a la altura de crucero $z_{\text{target}} = 2,5$ metros.
- **GOTO_START**: Vuela directo hacia el punto inicial de su franja de barrido (W_0).
- **WAIT_SYNC**: Al llegar a W_0 , se queda en vuelo estacionario (*hover*) y publica su estado en el tópico `/patrol_sync`.
- **PATROL**: Cuando comprueba que los N drones del enjambre están listos en el tópico, empieza a patrullar en paralelo siguiendo su trayectoria.
- **FINISHED**: Al terminar el barrido, el dron vuelve a su punto de despegue y aterriza apagando los motores.

4.4.3. Clasificación y Flujo de Comunicación Multi-Robot

Para analizar la coordinación del enjambre utilizando los conceptos vistos en clase, la interacción entre los drones se clasifica de la siguiente manera:

1. **Comunicación Explícita (Directa):** Se produce cuando los drones se envían mensajes directos de datos por la red para coordinarse:
 - **Sincronización de inicio (/patrol_sync):** Durante la espera en vuelo estacionario (WAIT_SYNC), cada dron publica su estado "listo" (mediante un mensaje `String`). Todos leen este tópico para hacer un quórum (comprobación de que están los N drones listos) antes de empezar a patrullar a la vez. Este mecanismo actúa como un protocolo de confirmación o *handshake* digital, muy similar al utilizado mediante señales físicas de I/O en la práctica 2.
 - **Envío de detecciones (/detections_georeferenced):** Cada dron transmite su lista de basura georreferenciada (`Detection2DArray`) directamente al nodo central `map_consolidator.py` para actualizar el mapa.
2. **Comunicación Implícita (Indirecta o Estigmergia):** Se da cuando los robots obtienen información del entorno o de los compañeros sin necesidad de enviarse mensajes directos por la red:
 - **Sensores Locales:** Los drones utilizan su propia cámara, GPS y altímetro para tomar medidas locales de forma individual y autónoma, permitiéndoles navegar sin sobrecargar el canal de comunicaciones ni depender de una red centralizada para el vuelo básico.
 - **Estigmergia Pasiva:** En robótica de enjambres, la stigmergia representa la comunicación indirecta a través de la modificación del medio físico (como las feromonas de las hormigas). En nuestro proyecto, los drones cooperan de forma indirecta registrando sus hallazgos en un archivo común compartido (`mapa_residuos.csv`). De este modo, los drones no necesitan comunicarse entre sí las posiciones de basura detectadas; simplemente modifican el "mapa del entorno común" el consolidador unifica los datos.
3. **Paradigma de Coordinación (CTDE):** El enjambre sigue el paradigma de **Entrenamiento Centralizado con Ejecución Descentralizada** (*Centralized Training with Decentralized Execution - CTDE*). En nuestro caso, la consolidación de los datos es centralizada (el nodo central recoge todo y crea el CSV global), pero el guiado y el vuelo de cada dron es completamente descentralizado y autónomo. Esto hace que el sistema sea muy robusto, ya que si un dron falla, los demás siguen volando y cubriendo sus zonas sin verse afectados.



4.5. Visión Artificial y Posicionamiento GPS

El procesamiento de imágenes y mapa se realiza en el paquete `uav_vision`. Este paquete se encarga de procesar el vídeo de la cámara, detectar la basura, calcular su posición en coordenadas reales y guardarlo en el mapa final.

4.5.1. Detección de Objetos en Tiempo Real (YOLOv8)

En el nodo `yolo_detector.py` nos suscribimos al tópico de la cámara del dron (`/uav_i/camera/image_raw`). Su funcionamiento es el siguiente:

1. Recibe la imagen de la cámara de ROS y la convierte a una matriz OpenCV usando la librería `CvBridge` para poder procesar la imagen con código de Python.
2. Le pasamos la imagen a la red neuronal YOLOv8 utilizando el modelo preentrenado `taco.pt` (entrenado con el dataset de basura *TACO*), usando un umbral de confianza de 0,20 para detectar objetos.
3. Para cada trozo de basura detectado, calculamos el centro de su recuadro en píxeles (u, v) , lo metemos en un mensaje de tipo `Detection2DArray` y lo publicamos en el tópico `/uav_i/yolo/detections`.



Figura 6: Inferencia en tiempo real del flujo de vídeo usando YOLOv8 sobre objetos de la simulación.

4.5.2. Motor Matemático de Georreferenciación

El nodo `georeferencer_node.py` se encarga de calcular la posición real en coordenadas GPS (latitud y longitud) a partir de los píxeles (u, v) de la imagen. La lógica matemática sigue estos tres pasos:

1. **Cálculo de la Distancia Focal en Píxeles (f_{px}):** Usando la anchura de la imagen (640 píxeles) y el campo de visión horizontal de la cámara ($HFOV = 1,089$ radianes), calculamos la distancia focal virtual en píxeles:

$$f_{px} = \frac{640}{2 \cdot \tan\left(\frac{HFOV}{2}\right)} \quad (4)$$

2. **Proyección a Coordenadas Métricas del Dron** (d_x, d_y): Calculamos el desvío en píxeles de la detección respecto al centro de la imagen ($u_c, v_c = 320, 240$):

$$\Delta u = u - 320 \quad (5)$$

$$\Delta v = v - 240 \quad (6)$$

Y usando la altura (h) medida por el altímetro, estimamos la distancia en metros de la basura respecto al dron:

$$d_y = \frac{\Delta u \cdot h}{f_{px}} \quad (7)$$

$$d_x = \frac{\Delta v \cdot h}{f_{px}} \quad (8)$$

donde d_x es la distancia hacia adelante/atrás y d_y es la distancia lateral hacia los lados en el eje del dron.

3. **Conversión Geodésica a Grados (WGS84 local)**: Con la posición GPS actual del dron ($\text{lat}_{\text{drone}}, \text{lon}_{\text{drone}}$) y sabiendo que un grado terrestre equivale a unos 111 132 metros (R_{deg}), sumamos el desplazamiento en metros para obtener la posición GPS exacta de la basura:

$$\text{lat}_{\text{target}} = \text{lat}_{\text{drone}} + \frac{d_x}{R_{\text{deg}}} \quad (9)$$

$$\text{lon}_{\text{target}} = \text{lon}_{\text{drone}} + \frac{d_y}{R_{\text{deg}} \cdot \cos\left(\frac{\pi \cdot \text{lat}_{\text{drone}}}{180}\right)} \quad (10)$$

$$\text{alt}_{\text{target}} = h \quad (11)$$

Esta posición calculada se publica en el tópic `detections.georeferenced` dentro del namespace de cada dron.

4.5.3. Consolidación Global y Filtrado de Duplicados

El nodo central `map consolidator.py` escucha las posiciones GPS de todos los drones a la vez. Cuando le llega una nueva detección $P_{\text{new}} = (\text{lat}_{\text{new}}, \text{lon}_{\text{new}})$, para no guardar la misma basura varias veces (si la ven varios drones o el mismo dron en fotos seguidas), calculamos la distancia en metros a las basuras ya registradas usando la **Fórmula del Haversine**:

$$\Delta\phi = \frac{\pi}{180}(\text{lat}_{\text{new}} - \text{lat}_j) \quad (12)$$

$$\Delta\lambda = \frac{\pi}{180}(\text{lon}_{\text{new}} - \text{lon}_j) \quad (13)$$

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\left(\frac{\pi \cdot \text{lat}_{\text{new}}}{180}\right) \cos\left(\frac{\pi \cdot \text{lat}_j}{180}\right) \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (14)$$

$$c = 2 \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a}) \quad (15)$$

$$d = R_{\text{Earth}} \cdot c \quad (16)$$

donde $R_{\text{Earth}} = 6\,371\,000$ metros es el radio medio de la Tierra.

Si la distancia calculada es menor de 2,0 metros (d_{min}), consideramos que es el mismo objeto y lo descartamos. Si es mayor, lo guardamos en memoria, lo escribimos con su marca de tiempo en el archivo `mapa_residuos.csv` y redibujamos el mapa dinámico en `mapa_residuos.png` (Figura 7) de forma asíncrona mediante Matplotlib.

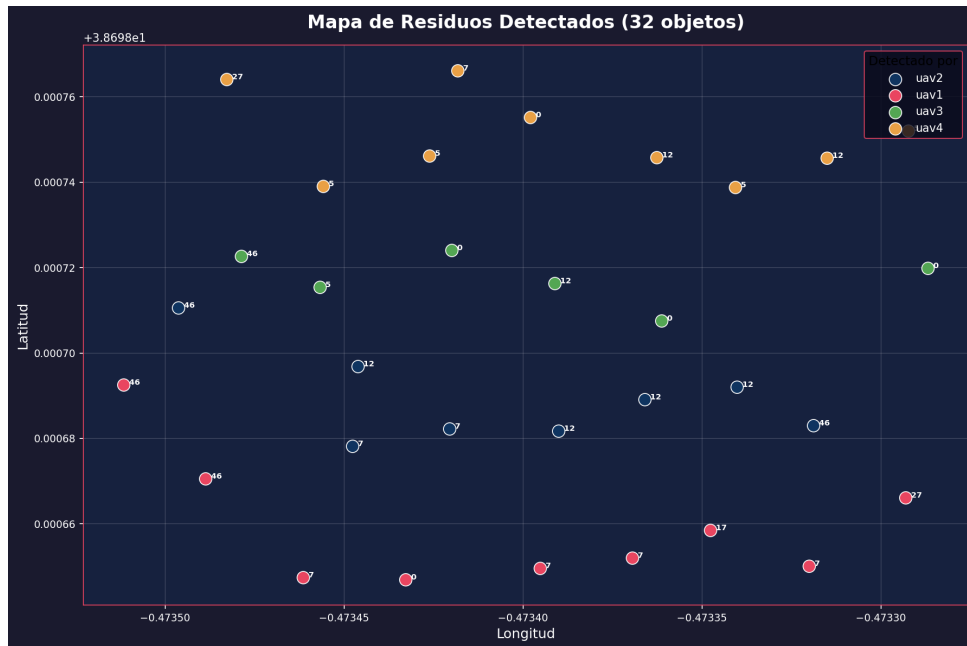


Figura 7: Mapa generado automáticamente en Matplotlib indicando la tipología de residuo y el dron detector.

5. Estructura del Sistema de Ficheros

El proyecto está organizado en varios paquetes de ROS 2 para que el código quede más ordenado y sea más fácil de gestionar:

uav_gazebo/ Contiene el mundo de simulación y los archivos launch para lanzar todo en Gazebo.

- **worlds/outdoor.world:** Define el entorno de la playa, la iluminación y coloca los modelos 3D de la basura.
- **launch/multi_uav.launch.py:** Lanza los drones en Gazebo aplicando los namespaces correspondientes para que no haya conflictos.

uav_description/ Contiene los archivos para definir cómo es el dron físicamente y cómo se visualiza.

- **urdf/*:** Archivos XACRO que definen el chasis, los motores y los sensores del dron.
- **rviz/uav.rviz:** Configuración de RViz2 para ver el modelo del dron de forma limpia.

uav_control/ Controla el vuelo y comportamiento del enjambre de drones.

- **uav_control/patrol.py:** Nodo que lleva la máquina de estados, el guiado de los drones y el barrido coordinado de la playa.

uav_vision/ Se encarga del procesamiento de imágenes y la creación del mapa de residuos.

- **yolo_detector.py:** Nodo que coge las imágenes de la cámara, las pasa por YOLOv8 y publica las detecciones.
- **georeferencer_node.py:** Nodo que calcula la posición GPS real de la basura a partir de los píxeles de la imagen.
- **map_consolidator.py:** Nodo central que elimina los duplicados con la fórmula del Haversine y guarda el mapa final en CSV e imagen PNG.

6. Dificultades y Resoluciones

Durante el desarrollo del proyecto surgieron varios problemas y dificultades que tuvimos que ir resolviendo:

1. **Detección de objetos 3D con YOLOv8 (Sim2Real):** Al principio, el modelo de YOLOv8 (entrenado con fotos reales) no detectaba bien los objetos 3D de Gazebo porque eran muy lisos y no tenían texturas reales. Para solucionarlo, ajustamos el umbral de confianza (*confidence threshold*) a 0,20 para hacerlo más sensible y cambiamos la textura del suelo del simulador por una arena clara que contrastara más con la basura. De esta forma, la red neuronal pudo empezar a detectar los objetos por sus formas básicas sin confundirse con el suelo.
2. **Despegue y avance descompasado del enjambre:** Como cada dron tarda un tiempo ligeramente diferente en despegar e ir a su posición inicial, los primeros que llegaban empezaban a patrullar antes de que los demás estuvieran listos, desorganizando el barrido. Lo solucionamos creando un tópico común llamado `/patrol_sync` donde los drones publican su estado. Ninguno empieza a patrullar hasta comprobar en este tópico que todos los drones del enjambre están listos en sus puntos de salida.
3. **Ralentización del ordenador por las ventanas de vídeo (GUI):** Ejecutar tres nodos de visión a la vez (uno por dron) mostrando la imagen de la cámara en tiempo real con las cajas y textos de OpenCV consumía demasiados recursos, haciendo que el ordenador fuera lentísimo y se congelara la simulación. La solución fue añadir el parámetro `show_gui:=false` para ejecutar el procesamiento de visión de fondo sin abrir ventanas gráficas. Así, el procesamiento interno sigue funcionando exactamente igual pero el simulador va mucho más rápido y fluido.

7. Conclusiones y Trabajo Futuro

Como conclusión general, el proyecto demuestra que un enjambre de drones es muy útil para buscar y mapear residuos en una playa de forma automatizada. Al usar ROS 2 y Gazebo, hemos comprobado que si aumentamos el número de drones (N), el tiempo necesario para explorar toda la zona disminuye de forma proporcional. Además, al no haber un nodo o controlador central que maneje el vuelo, el sistema es mucho más seguro y robusto: si un dron tiene un problema y se cae, los demás pueden seguir patrullando sus zonas asignadas sin enterarse.

Como trabajos futuros o posibles mejoras para ampliar el proyecto, se proponen las siguientes ideas:

- **Procesamiento a bordo (Edge Computing):** En un escenario real, transmitir vídeo en alta definición por Wi-Fi o radio desde cada dron a un ordenador central consume mucho ancho de banda y da retrasos. Una mejora importante sería meter en cada dron una tarjeta pequeña de procesamiento (como una NVIDIA Jetson Nano o una Google Coral). Así, el algoritmo YOLOv8 correría directamente en el propio dron y solo tendríamos que enviar por red las coordenadas GPS del residuo detectado, que es un mensaje muy ligero.
- **Simulación con PX4 (SITL/HITL):** Sustituir el plugin de movimiento de Gazebo por una conexión con el simulador de controladora de vuelo de PX4. Esto permitiría simular el vuelo usando el mismo software que llevaría un dron real, lo que haría la simulación mucho más fiel a la realidad.
- **Reentrenamiento del modelo (Fine-tuning):** Sacar fotos de los objetos 3D directamente dentro del propio simulador de Gazebo para reentrenar el modelo de YOLOv8 con

ellas. Esto ayudaría a que la red neuronal reconozca los modelos virtuales sin ningún fallo antes de hacer pruebas de vuelo reales.

- **Recogida activa de basura:** Ir un paso más allá del simple mapeo y diseñar un sistema para recoger la basura. Se podría añadir un brazo o una pinza pequeña a los drones para que la recojan ellos mismos, o bien añadir robots terrestres al enjambre que reciban las coordenadas GPS enviadas por los drones y vayan por tierra a recoger los residuos.

8. Uso de Herramientas de IA

Durante el desarrollo del proyecto hemos utilizado Inteligencia Artificial como ChatGPT o Gemini para ayudarnos a depurar el código Python, configurar el XACRO de los drones en Gazebo Harmonic, y corregir la redacción y formato de esta memoria.

Referencias

- [1] Open Robotics. *Documentación Oficial de ROS 2 Jazzy Jalisco*. <https://docs.ros.org/en/jazzy/>
- [2] Open Source Robotics Foundation. *Gazebo Harmonic Documentation*. <https://gazebo.sim.org/docs/harmonic>
- [3] TACO - Trash Annotations in Context. *Repositorio de imágenes e identificadores de residuos*. <http://tacodataset.org/>
- [4] Ultralytics. *YOLOv8 Architecture and Object Detection*. <https://docs.ultralytics.com/>
- [5] Wikipedia. *Fórmula del Haversine - Cálculo de distancias geodésicas*. https://es.wikipedia.org/wiki/F%C3%B3rmula_del_haversine